

ASSIGNMENT NO.6: Data Manipulation and Table Management

Objective:

The objective of this practical is to help students understand how to manage table data in MySQL. Students will learn how to insert records using INSERT, update records using UPDATE, delete records using DELETE, and apply important constraints such as PRIMARY KEY, FOREIGN KEY, and NOT NULL.

Introduction to Data Manipulation and Table Management

Data manipulation means adding, changing, and deleting records from a database table. In MySQL, data manipulation is mainly performed using INSERT, UPDATE, and DELETE commands.

Table management means creating tables with proper structure and rules. These rules are called constraints. Constraints help maintain accuracy, uniqueness, and relationship between tables.

Common Data Manipulation Commands

Command	Purpose	Example Use
INSERT	Adds new records	Add new student details
UPDATE	Modifies existing records	Change student city or marks
DELETE	Removes records	Delete inactive student record

Common Constraints in MySQL

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each record	StudentID
FOREIGN KEY	Creates relationship between tables	CourseID in Student table
NOT NULL	Does not allow empty values	StudentName

Creating Database and Tables with Constraints

In this practical, we will create a database named CollegeDB. We will create two related tables: Course and Student.

The Course table stores course details. The Student table stores student details and connects each student with a course using CourseID.

Course Table Structure

Column Name	Data Type	Constraint	Description
CourseID	INT	PRIMARY KEY	Unique ID for each course
CourseName	VARCHAR(50)	NOT NULL	Name of the course
Fees	DECIMAL(10,2)	NOT NULL	Course fees

Student Table Structure

Column Name	Data Type	Constraint	Description
StudentID	INT	PRIMARY KEY	Unique ID for each student
StudentName	VARCHAR(50)	NOT NULL	Name of the student
Age	INT		Age of the student
City	VARCHAR(30)		City of the student
Column Name	Data Type	Constraint	Description
CourseID	INT	FOREIGN KEY	Refers to CourseID in Course table

MySQL Code

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
CREATE TABLE Course (CourseID INT PRIMARY KEY, CourseName VARCHAR(50) NOT
NULL, Fees DECIMAL(10,2) NOT NULL );
CREATE TABLE Student (StudentID INT PRIMARY KEY, StudentName VARCHAR(50)
NOT NULL, Age INT, City VARCHAR(30), CourseID INT, FOREIGN KEY (CourseID)
REFERENCES Course(CourseID) );
```

Explanation

SQL Part	Purpose
<code>`CREATE DATABASE CollegeDB;`</code>	Creates a new database
<code>`USE CollegeDB;`</code>	Selects the database
<code>`PRIMARY KEY`</code>	Ensures unique values in ID column
<code>`NOT NULL`</code>	Ensures value cannot be left blank
<code>`FOREIGN KEY`</code>	Connects Student table with Course table
<code>`REFERENCES Course(CourseID)`</code>	Links Student.CourseID with Course.CourseID

3. Inserting Data using INSERT

The ``INSERT`` command is used to add new records into a table. Before inserting student records, course records should be inserted first because the ``Student`` table depends on the ``Course`` table through a foreign key.

Insert Records into Course Table

```
INSERT INTO Course
(CourseID, CourseName, Fees)
VALUES
(101, 'BSc IT', 45000.00),
(102, 'BCA', 40000.00),
(103, 'BSc CS', 42000.00),
(104, 'Data Science', 55000.00);
```

Insert Records into Student Table

```
INSERT INTO Student (StudentID, StudentName, Age, City, CourseID) VALUES
(1, 'Rahul Sharma', 20, 'Mumbai', 101), (2, 'Priya Patil', 19, 'Pune',
102), (3, 'Aman Verma', 21, 'Nashik', 103),
(4, 'Sneha Joshi', 20, 'Mumbai', 101), (5, 'Karan Mehta', 22, 'Thane',
```

```
102), (6, 'Neha Shah', 19, 'Pune', 104);
```

Course Table Output

CourseID	CourseName	Fees
101	BSc IT	45000.00
102	BCA	40000.00
103	BSc CS	42000.00
104	Data Science	55000.00

Student Table Output

StudentID	StudentName	Age	City	CourseID
1	Rahul Sharma	20	Mumbai	101
2	Priya Patil	19	Pune	102
3	Aman Verma	21	Nashik	103
4	Sneha Joshi	20	Mumbai	101
5	Karan Mehta	22	Thane	102
6	Neha Shah	19	Pune	104

Explanation

Point	Description
Course records inserted first	Because Student table uses CourseID as foreign key
Student records inserted later	Each student is assigned a valid CourseID
Invalid CourseID not allowed	MySQL will reject a Student record if CourseID does not exist in Course table

Updating Records using UPDATE

The UPDATE command is used to modify existing records in a table. It is commonly used when there is a change in student details, course fees, city, age, or any other stored value.

Syntax

```
UPDATE table_name SET column_name = new_value WHERE condition;
```

Example 1: Update Student City

```
UPDATE Student SET City = 'Mumbai' WHERE StudentID = 2;
```

Output After Update

StudentID	StudentName	Age	City	CourseID
2	Priya Patil	19	Mumbai	102

Example 2: Update Course Fees

```
UPDATE Course SET Fees = 47000.00 WHERE CourseID = 101;
```

Output After Update

CourseID	CourseName	Fees
101	BSc IT	47000.00

Important Note

Always use WHERE with UPDATE. If WHERE is not used, all records in the table may get updated.

Example of Wrong Query

```
UPDATE Student SET City = 'Mumbai';
```

Problem

Issue	Explanation
Missing WHERE clause	Updates city of all students
Data loss risk	Original city values may be overwritten
Poor practice	Should be avoided in practical work

5. Deleting Records using DELETE

The DELETE command is used to remove records from a table. It should be used carefully because deleted records may not be recovered unless backup is available.

Syntax

```
DELETE FROM table_name WHERE condition;
```

Example: Delete One Student Record

```
DELETE FROM Student WHERE StudentID = 6;
```

Output After Delete

StudentID	StudentName	Age	City	CourseID
1	Rahul Sharma	20	Mumbai	101
2	Priya Patil	19	Mumbai	102
3	Aman Verma	21	Nashik	103
4	Sneha Joshi	20	Mumbai	101
5	Karan Mehta	22	Thane	102

Important Note

Always use WHERE with DELETE. If WHERE is not used, all records will be deleted from the table.

Example of Wrong Query

```
DELETE FROM Student;
```

Problem

Issue	Explanation
Missing WHERE clause	Deletes all records from Student table
High risk	Complete table data will be removed
Should be avoided	Use condition carefully

6. Applying PRIMARY KEY, FOREIGN KEY, and NOT NULL Constraints

Constraints are rules applied to table columns. They help maintain valid, accurate, and reliable data.

Constraint Summary

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each row	StudentID
FOREIGN KEY	Connects one table to another table	CourseID
NOT NULL	Does not allow blank values	StudentName

Example 1: PRIMARY KEY Constraint

```
CREATE TABLE Department (DepartmentID INT PRIMARY KEY, DepartmentName  
VARCHAR(50) NOT NULL );
```

Explanation

Column	Constraint	Meaning
DepartmentID	PRIMARY KEY	Duplicate values not allowed
DepartmentName	NOT NULL	Department name cannot be empty

Example 2: FOREIGN KEY Constraint

```
CREATE TABLE Faculty (FacultyID INT PRIMARY KEY, FacultyName VARCHAR(50) NOT  
NULL, DepartmentID INT, FOREIGN KEY (DepartmentID) REFERENCES  
Department(DepartmentID) );
```

Explanation

Column	Constraint	Meaning
FacultyID	PRIMARY KEY	Unique faculty ID
FacultyName	NOT NULL	Faculty name is required
DepartmentID	FOREIGN KEY	Must match DepartmentID from Department table

Example 3: NOT NULL Constraint

```
CREATE TABLE AdminUser (UserID INT PRIMARY KEY, UserName VARCHAR(50) NOT  
NULL, Email VARCHAR(60) NOT NULL );
```

Explanation

Column	Constraint	Meaning
UserName	NOT NULL	Username must be entered
Email	NOT NULL	Email cannot be left blank

Final Summary Table

Concept	MySQL Command/Constraint	Purpose
Insert data	INSERT INTO	Adds new records
Update data	UPDATE	Modifies existing records
Delete data	DELETE	Removes records
Unique record	PRIMARY KEY	Prevents duplicate ID
Table relationship	FOREIGN KEY	Links two related tables
Required value	NOT NULL	Prevents empty values

Safe update/delete	WHERE	Applies change to selected records only
--------------------	-------	---

Questions:

Q1) Create a MySQL database named CollegeDB and create two tables: Course and Student.

Course Table

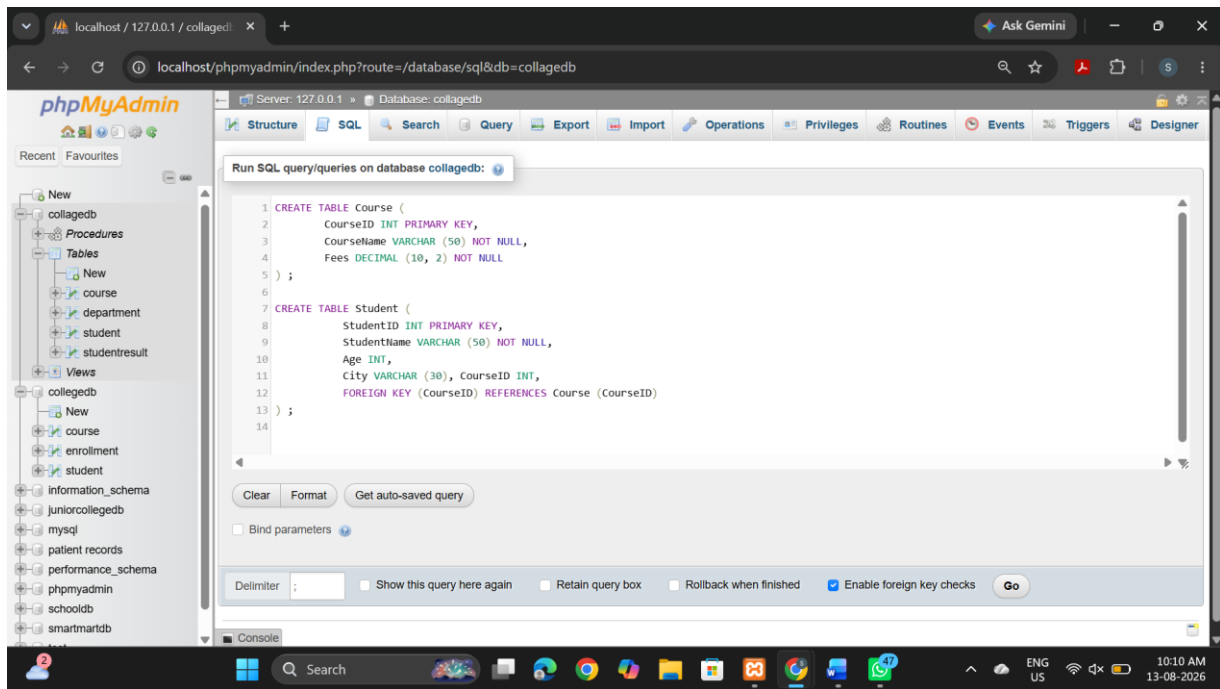
Column Name	Data Type	Constraint
CourseID	INT	PRIMARY KEY
CourseName	VARCHAR(50)	NOT NULL
Fees	DECIMAL(10,2)	NOT NULL

Student Table

Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY
StudentName	VARCHAR(50)	NOT NULL
Age	INT	
City	VARCHAR(30)	
CourseID	INT	FOREIGN KEY

```
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR (50) NOT NULL,
    Fees DECIMAL (10, 2) NOT NULL
);
```

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR (50) NOT NULL,
    Age INT,
    City VARCHAR (30), CourseID INT,
    FOREIGN KEY (CourseID) REFERENCES Course (CourseID)
);
```



Q2) Insert at least 4 records into the Course table and 6 records into the Student table.
Make sure every student has a valid CourseID from the Course table.

Query

INSERT INTO Course (CourseID, CourseName, Fees)

VALUES

(101, 'BSC IT', 45000.00)
(102, 'BCA', 40000.00),
(103, 'BSc CS', 42000.00),
(104, 'Data Science', 55000.00);

INSERT INTO Student

(StudentID, StudentName, Age, City, CourseID)

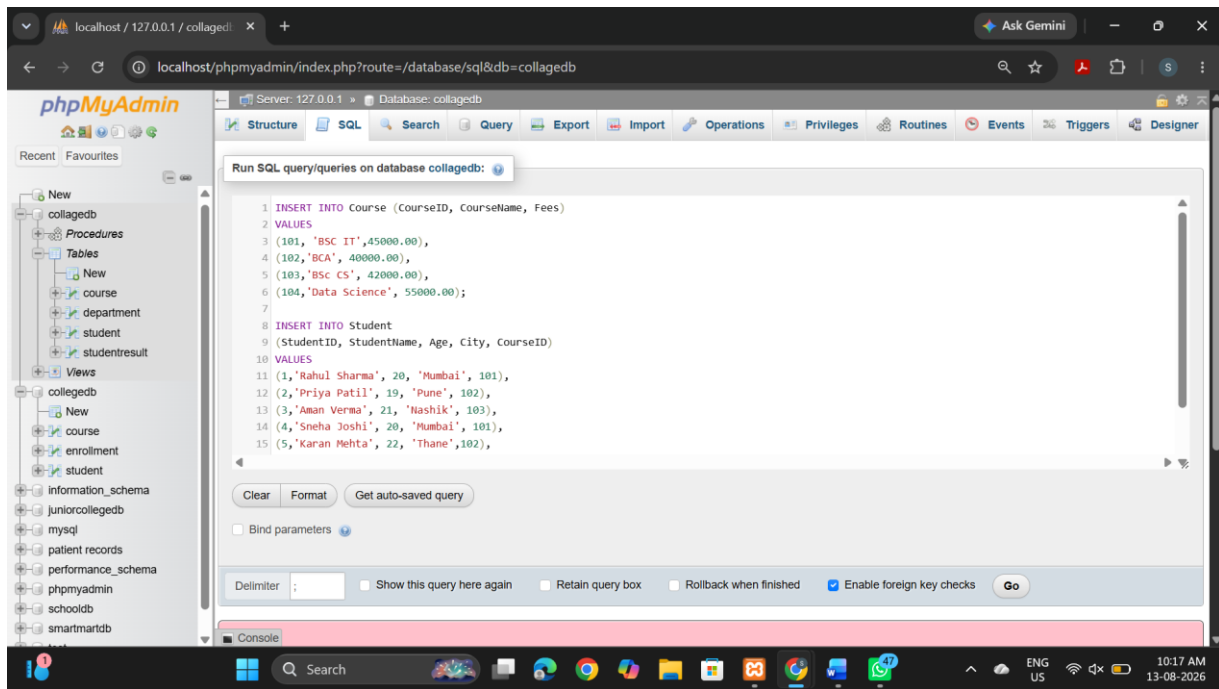
VALUES

(1, 'Rahul Sharma', 20, 'Mumbai', 101),
(2, 'Priya Patil', 19, 'Pune', 102),
(3, 'Aman Verma', 21, 'Nashik', 103),
(4, 'Sneha Joshi', 20, 'Mumbai', 101),
(5, 'Karan Mehta', 22, 'Thane', 102),
(6, 'Neha Shah', 19, 'Pune', 104);

To verify:

SELECT * FROM Course;

SELECT * FROM Student;



Q3) Write a MySQL query to insert one new student record into the Student table.

The record should include:

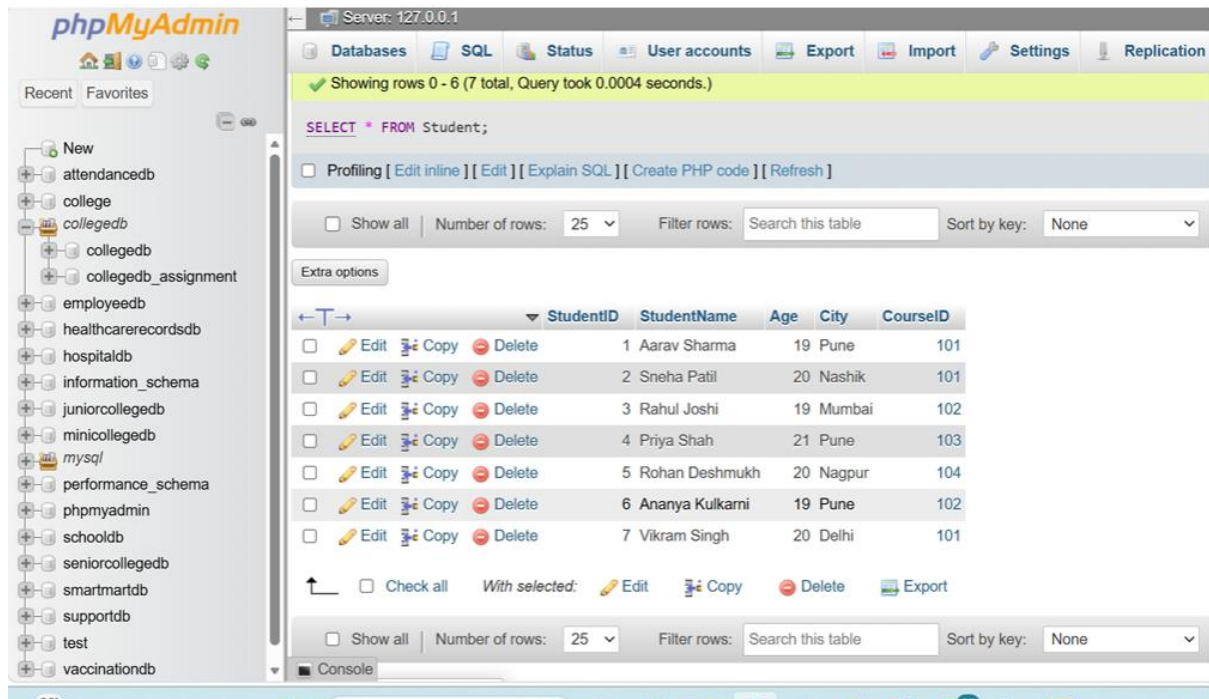
Field
StudentID
StudentName
Age
City
CourseID

INSERT INTO Student (StudentID, StudentName, Age, City, CourseID)

VALUES (7, 'Vikram Singh', 20, 'Delhi', 101);

VERIFY:

Select * from student



Q4) Write a MySQL query to update the city of a student using StudentID.

Example requirement:

Change Required

Update the city of StudentID 2 to Mumbai

UPDATE Student

SET City = 'Mumbai'

WHERE Student ID = 2;

Verify:

SELECT * FROM Student

WHERE StudentID = 2;

The top screenshot shows the phpMyAdmin interface with the 'collegedb' database selected. The SQL query box contains the following code:

```
USE CollegedB;

[ Edit inline ] [ Edit ] [ Create PHP code ]
```

The console output shows: "MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)"

The bottom screenshot shows the same interface, but with an error message: "Error: #1046 No database selected". Below the error, the SQL query box contains the following code:

```
UPDATE Student SET City = 'Mumbai' WHERE StudentID = 2;

[ Edit inline ] [ Edit ] [ Create PHP code ]
```

The console output shows: "1 row affected. (Query took 0.0111 seconds.)"

Below the console output, the 'Student' table is displayed with the following data:

StudentID	StudentName	Age	City	CourseID
2	Sneha Patil	20	Mumbai	101

Q5) Write a MySQL query to update the fees of a course using CourseID.

Example requirement:

Change Required

Update the fees of CourseID 101 to 47000

UPDATE Course

SET Fees = 47000.00

WHERE CourseID = 101;

Verify:

SELECT * FROM Course

WHERE CourseID = 101;

The screenshot shows the phpMyAdmin interface with the 'collegedb' database selected. The SQL query entered is `SELECT * FROM Course WHERE CourseID = 101;`. The result shows one row: CourseID 101, CourseName 'B.Sc. Data Science', and Fees 47000.00. The interface includes a sidebar with database listings, a top navigation bar, and a bottom console area.

Q6) Write a MySQL query to delete one student record using StudentID.

Also write one line explaining why the WHERE clause is important while using DELETE.

DELETE FROM Student

WHERE StudentID = 6;

Verify:

SELECT * FROM Student;

The screenshot shows the phpMyAdmin interface with the 'collegedb' database selected. The SQL query entered is `Select * from STUDENT;`. The result shows a table with 7 rows of student records. The interface includes a sidebar with database listings, a top navigation bar, and a bottom console area.

StudentID	StudentName	Age	City	CourseID
1	Aarav Sharma	19	Pune	101
2	Sneha Patil	20	Mumbai	101
3	Rahul Joshi	19	Mumbai	102
4	Priya Shah	21	Pune	103
5	Rohan Deshmukh	20	Nagpur	104
7	Vikram Singh	20	Delhi	101

The WHERE clause ensures that only the specified student record is deleted.
Without WHERE, all records from the Student table would be deleted.

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well Done []

Signature of Instructor: _____